

16

Máquinas de Turing

A necessidade de formalizar os processos algorítmicos levou, nas décadas 20 e 30 do século XX, a diversos estudos, entre os quais os de Post, Church e Turing, com vista a estudo formal da potência de tais processos. Apesar dos modelos então criados serem equivalentes, nesta abordagem vamos sempre utilizar o modelo das Máquinas de Turing, como suporte para as matérias estudadas.

16.1 *Máquinas de Turing*

O modelo de Máquina de Turing que aqui vamos usar é constituído por uma unidade de controlo, que contém o “programa” da máquina, uma cabeça de escrita e leitura, e uma fita que vamos supor ilimitada e totalmente preenchida por caracteres "espaço" $_$. No início da execução da máquina, o input está escrito na fita, a cabeça está colocada sobre o primeiro carácter do input e o controlo da máquina encontra-se num estado que vamos designar de inicial. Em cada passo, dependendo do carácter que pode ser lido pela cabeça e do estado em que o controlo se encontra, o carácter sob a cabeça pode ser mudado, o estado do controlo alterado e a cabeça movida, uma posição, para esquerda ou direita. Se em alguma altura o controlo chegar a um estado especial de aceitação ou de rejeição, pára e comunica para o exterior se aceita ou rejeita de acordo com o estado.

No modelo de máquina de Turing que aqui vamos usar, estabelecemos como regra que a cabeça da máquina não se pode deslocar para a esquerda da posição inicial. Isso significa que com a cabeça na posição inicial, se o controlo indicar que a mesma se deve mover para a esquerda, a cabeça mantém-se imóvel. Na realidade esta restrição não tem qualquer importância como veremos mais à frente. Formalmente, uma máquina de Turing é definida da seguinte forma.

Definição 16.1 (Máquina de Turing) *Um máquina de Turing (TM), é um túbulo*

$$\langle Q, \Sigma, \Gamma, \delta, q_0, q_y, q_n \rangle,$$

em que

- Q é o conjunto finito de estados;
- Σ é o alfabeto de input;

- Γ é o alfabeto da fita;
- δ é a função de transição da máquina,

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{R, L\};$$

- q_0 é o estado inicial;
- q_y o estado de aceitação;
- q_n o estado de rejeição.

Configuração de uma TM O estado de uma TM fica completamente determinado sabendo o estado da máquina, o conteúdo da fita e a posição da cabeça. A este conjunto de informação chamamos configuração da máquina. Geralmente representamos uma configuração por

$$l_1 l_2 \cdots l_m q_i r_0 r_1 \cdots r_m$$

significando com isso que a TM se encontra no estado q_i , que a fita é

$$l_1 l_2 \cdots l_m r_0 r_1 \cdots r_m \cdots,$$

estando a cabeça sobre o carácter r_0 .

Funcionamento de uma TM Numa TM M dizemos que a configuração C_1 produz a configuração C_2 , e escrevemos $C_1 \vdash_M C_2$, se M puder evoluir de C_1 para C_2 de acordo com as suas regras de funcionamento, dadas por δ . Suponhamos $a, b, c \in \Gamma$, $u, v \in \Gamma^*$ e $q_i, q_j \in Q$. Nesse caso $u a q_i b v$ é uma possível configuração de M . Então

$$u a q_i b v \vdash u q_j a c v \text{ se } \delta(q_i, b) = (q_j, c, L),$$

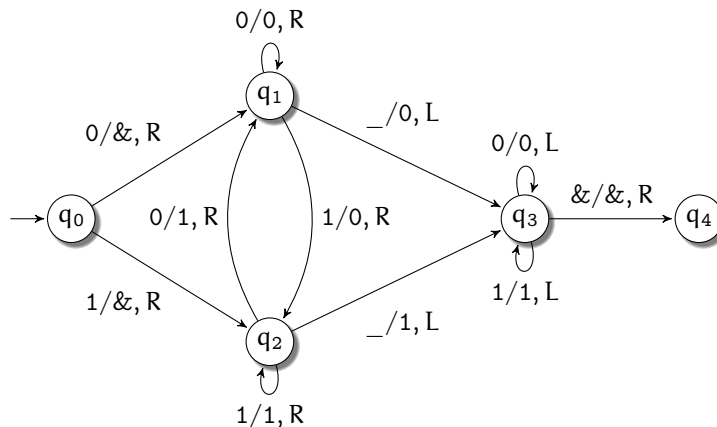
$$u a q_i b v \vdash u a c q_j v \text{ se } \delta(q_i, b) = (q_j, c, R).$$

A configuração inicial de M com input w é

$$q_0 w$$

ou seja o estado inicial é q_0 , a fita contém exactamente o input w e a cabeça da máquina encontra-se sobre o primeiro carácter do input.

O facto do modelo da TM não permitir que a sua cabeça se mova para a esquerda para da primeira posição da fita, pode ser facilmente ultrapassada, primeiro colocando um símbolo que marque essa posição com um carácter especial que não ocorra no resto da fita (por exemplo $\&$), e depois passando a detectar esse carácter como o extremo esquerdo da fita. Introduzir tal carácter no início da fita pode ser feito somente com 4 estados.



A partir do estado q_4 a TM passa a poder detectar o extremo esquerdo da fita pelo caracter $\&$, tendo à sua direita o input “intacto”.

Definição 16.2 A máquina de Turing $M = \langle Q, \Sigma, \Gamma, \delta, q_0, q_y, q_n \rangle$ aceita o input w se existe uma sequência de configurações $C_1, \dots, C_k$ em que:

- i) C_1 é a configuração inicial de M com input w ;
- ii) para qualquer i , $C_i \vdash_M C_{i+1}$;
- iii) C_k é uma configuração de aceitação, ou seja o estado por ela referido é o estado q_y de M .

A linguagem das palavras que são aceites pela TM M diz-se a linguagem de M , e designa-se por $\mathcal{L}(M)$.

Linguagens reconhecíveis

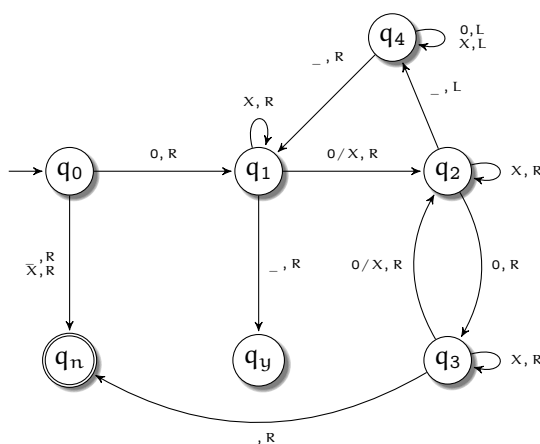
Definição 16.3 Uma linguagem L diz-se Turing reconhecível, reconhecível, ou recursivamente enumerável, se existir uma TM M tal que $\mathcal{L}(M) = L$, ou seja se há uma TM que a reconhece. De M diz-se ser o reconhecedor de L .

Linguagens decidíveis Uma TM M que, pára para todos os inputs, e reconhece a linguagem L diz-se que decide a linguagem L .

Definição 16.4 Uma linguagem L diz-se decidível, ou recursiva, se existir uma TM M que decida L . Nesse caso diz-se que M é um decisor de L .

Claro que se uma linguagem é decidível também é reconhecível.

Exemplo 16.5 Seja $L = \{0^{2^n} : n \geq 0\}$. A TM M representada pelo diagrama seguinte decide L . Neste diagrama, quando uma transição se encontra marcada como X, R isso significa que lendo um X se move a cabeça para a direita e que o símbolo da fita é deixado inalterado. Se a transição está marcada como $X/Y, R$ isso significa que com a leitura de um X , se substitui o símbolo por um Y na fita, e se move a cabeça para a direita.



A ideia do “programa” é de ir sempre marcando metade dos 0 existentes e garantir que o número dos que sobra é par.

Exercícios

Exercício 78 Mostrar que a linguagem $L = \{w\#w : w \in \{0, 1\}^*\}$ é decidível.

Exercício 79 Escrever uma TM que decida a linguagem $\{a^i b^j c^k : i \times j = k, i, j, k \geq 1\}$.

Exercício 80 Escrever uma TM que decida

$$\{\#x_1\#x_2\#\cdots\#x_l : x_i \in \{0, 1\}^* \wedge \forall i \neq j, x_i \neq x_j\}.$$

Exercício 81 Escrever uma TM que decida $\{m\#n : \text{com } m = n + 1, \text{ escritos em base } 5\}$.

Variantes de TM Por vezes definimos máquinas de Turing que não possuem somente uma fita mas várias. Para cada uma das fitas a máquina tem uma cabeça, que se pode mover independentemente. Normalmente, neste tipo de máquinas, como há mais do que uma fita, não se obriga as cabeças a moverem-se, podendo ficar na mesma posição depois da leitura e escrita. A função de transição de uma máquina assim definida deverá ser algo como

$$\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k.$$

Estes modelos são, no entanto, equivalentes à TM definida atrás.

Não é difícil entender que uma TM A com várias fitas possa simular uma TM B com uma só fita. Basta que simule o comportamento da máquina A na primeira fita e simplesmente não use as restantes fitas. A simulação no sentido contrário é mais complicada. Suponhamos, então, que A tem k fitas e vamos simular o efeito dessas k fitas guardando a informação correspondente na única fita de B. Para isso usamos um símbolo especial #, que adicionamos ao alfabeto Γ , como separador entre os conteúdos das diversas fitas. Para além do conteúdo das fitas, necessitamos também guardar a posição da cabeça da máquina A em cada uma das fitas. Para isso, para cada carácter a do alfabeto da fita de A, acrescentamos um novo carácter a' ao alfabeto da fita de B.

B Input: $w = w_1 w_2 \cdots w_n$

- 1: Primeiro inicializa-se a fita por forma a simular as k fitas de A. A fita deve ter a seguinte configuração:

$$\#w'_1 w_2 \cdots w_n \# _ ' \# _ ' \# \cdots \# _ ' \#$$

- 2: Para simular cada passo de A, percorre a fita desde o primeiro #, que marca o início da fita, até ao $(k+1)$ -ésimo # para determinar os símbolos sob as diversas cabeças virtuais. Seguidamente actualiza “as fitas” segundo o que está estipulado pela função de transição de A.
- 3: Se em algum momento B move uma cabeça virtual para a cima de um # isso significa que A moveria a correspondente cabeça para uma célula ainda não escrita ($_$) à direita da respectiva fita. Assim B escreve $_$ nessa posição e desloca todos os símbolos da fita uma posição para a direita. Depois disto continua a simulação como antes.

Como as fitas só são ilimitadas à direita o preceito 3 é suficiente como dispositivo para aumentar as fitas virtuais em B.

Máquinas de Turing não determinísticas

Da mesma forma que para os Autómatos Finitos (pag. 62) ou para os Autómatos de Pilha (pag. 177) as TM também podem ter uma variante não determinística. A sua definição é idêntica à da Definição 16.1 mas a função de transição é, agora,

$$\delta : Q \times \Gamma \rightarrow 2^{Q \times \Gamma \times \{L, R\}}.$$

A linguagem reconhecida por uma TM não determinística continua a ter a mesma definição: é o conjunto das palavras para as quais há uma sequência de configurações $C_1 C_2 \dots C_n$, da máquina, tais que C_1 é a configuração inicial tendo a referida palavra como input; para cada i , $C_i \vdash C_{i+1}$ e C_n é uma configuração de aceitação. Podemos imaginar que a TM executa paralelamente todas as possíveis computações, aceitáveis pela função de transição, a partir da configuração inicial e se algum ramo da computação chegar ao estado de aceitação, a TM pára e aceita o input.

Contrariamente ao que acontece com os PDA, as TM não determinísticas não são mais expressivas que as suas versões determinísticas. De facto, vamos mostrar que qualquer TM não determinística pode ser simulada por uma TM determinística equivalente.

Comecemos por observar que o número de alternativas não determinísticas que uma TM pode ter, dado um estado e um símbolo a ser lido na fita, é limitado por uma constante. Para isso basta observar que para cada estado q e símbolo γ a ser lido da fita, $\delta(q, \gamma) \subseteq Q \times \Gamma \times \{R, L\}$. Pelo que o número de alternativas é, no máximo, $m = 2nk$, com $n = |Q|$ e $k = |\Gamma|$.

Enquanto a execução de uma TM com um dado input, pode ser vista como uma sequência linear de configurações $C_1 C_2 \dots C_n \dots$ começada com a configuração inicial C_1 para aquele input, para uma TM não determinística a execução tem que ser vista como uma árvore, com raiz na configuração inicial para o input, e em que cada nó é uma configuração sendo os filhos as possíveis alternativas para o evoluir da execução. Podemos sentir tentados a simular a TM não determinística, percorrendo esta árvore com uma pesquisa em profundidade. Mas mesmo para uma palavra aceite pela máquina, nada nos garante que não haja um ramo de computação que não termine que ocorra antes do ramo que aceita o input, impedindo a simulação de alguma vez terminar. Temos, pois de proceder mais prudentemente. Se considerarmos uma ordem para o conjunto $Q \times \Gamma \times \{R, L\}$ podemos atribuir um endereço único a cada nó dessa árvore. Os endereços podem ser palavras de $\mathbb{N}^*$, em que ε representa a raiz (a configuração inicial) e um endereço $a_1 a_2 \dots a_{n-1} a_n$ representa a computação que no nó $a_1 a_2 \dots a_{n-1}$ “opta” por seguir a a_n -ésima possibilidade de computação. Observe-se que a sequência crescente dos inteiros escrita na base m corresponde aos endereços obtidos ao percorrer todos os nós da árvore com uma descida em largura. Podemos então usar este facto para dirigir a execução da simulação.

Estamos então em condições de provar o seguinte teorema.

Teorema 16.6 *Para qualquer TM não determinística existe uma TM determinística que a simula.*

Dem. Seja A uma TM não determinística. Vamos construir uma TM B com 3 fitas que simula A . Como vimos atrás (pag. 206) o número de fitas que a TM tem é irrelevante, pois todos os modelos são equivalentes, mas simplifica a exposição considerar este modelo em vez do modelo de uma só fita.

Para simplificar, chamemos às fitas F_1 , F_2 e F_3 . A fita F_1 vai ser usada somente para guardar o input, não sendo nunca modificada. A fita F_2 vai

conter o endereço de uma possível configuração na árvore de computação de A. A fita F3 será usada como fita de trabalho para cada uma das simulações a realizar.

B Input: w

- 1: A configuração inicial tem F1 contendo w , e as fitas F2 e F3 vazias.
- 2: Copia F1 para F3.
- 3: Usa F3 para simular A com input w num dos seus ramos de execução não determinística, do seguinte modo. Executa as instruções da máquina A usando os símbolos que vai lendo em F2 por forma de decidir quais as diversas “opções” que deve fazer no caso de haver uma escolha não determinística a fazer. Caso tenha chegado ao fim de F2 ou o endereço do nó não corresponder a uma execução possível passa para 4 ou se chegar a um estado de rejeição. Se chegar a um estado de aceitação, pára e aceita o input.
- 4: Substitui o conteúdo de F2 pelo inteiro seguinte em base m e volta para 2.

A simulação feita por B tem que recomeçar sempre em cada ramo de computação desde o início pois não há outra forma de repor a configuração correspondente ao nó anterior.

Se uma palavra w for aceite por A, isso significa que há um ramo de execução de A que chega a uma configuração C que contém o estado de aceitação de A. Essa configuração tem portanto um endereço na árvore de execução de A. Portanto ao cabo de um número finito de passos a máquina B acabará por ter esse endereço em F3 e a máquina B aceitará o input w . A direcção inversa é igualmente evidente. $\square$

Corolário 16.7 Uma linguagem L é reconhecível se e só se existir alguma TM não determinística que a reconheça.

17

Computabilidade

17.1 Linguagens reconhecíveis e decidíveis

Problemas decidíveis para Linguagens Regulares

Proposição 17.1 *Seja*

$$A_{\text{DFA}} = \{\langle A, w \rangle : A \text{ é um DFA e aceita } w\}.$$

A linguagem A_{DFA} é decidível.

Dem. Basta mostrar como construir um decisor M para A_{DFA} .

M **Input:** $\langle A, w \rangle$ (A DFA e w palavra)

- 1: Simula A com input w .
- 2: Se A aceitar w , para e **aceita**. Se A não aceitar w , para e **rejeita**.

De facto a escrita de M é bastante simples. Tem que verificar se as descrições que recebe de A e w são bem formadas, caso contrário rejeita. Para simular A tem somente que manter o estado em que a simulação de A se encontra e à medida que lê os símbolos de w proceder como descreve a função de transição de A . Quando a leitura de w termina, se o estado resultante constar da lista de estados finais, aceita, e rejeita no caso contrário. $\square$

Proposição 17.2 *Seja*

$$A_{\text{NFA}} = \{\langle A, w \rangle : A \text{ é um NFA e aceita } w\}.$$

A linguagem A_{NFA} é decidível.

Dem. O decisor M de A_{NFA} pode ser descrito como

M **Input:** $\langle A, w \rangle$ (A NFA e w palavra)

- 1: Transforma A num DFA A' .
- 2: Simula A' com input w .
- 3: Se A' aceitar w , para e **aceita**. Se A' não aceitar w , para e **rejeita**.

□

Proposição 17.3 *Seja*

$$A_{ER} = \{\langle \alpha, w \rangle : \alpha \text{ é uma expressão regular e } w \in \mathcal{L}(\alpha)\}.$$

A linguagem A_{ER} é decidível.

Proposição 17.4 *Seja*

$$E_{DFA} = \{\langle A \rangle : A \text{ é DFA e } \mathcal{L}(A) = \emptyset\}.$$

A linguagem E_{DFA} é decidível.

Dem. Podemos construir a TM M que decide E_{DFA} que percorre, com uma descida em largura, todos os estados de A , da seguinte forma

M **Input:** $\langle A \rangle$ (A DFA)

-
- 1: Marca o estado inicial de A .
 - 2: **repeat**
 - 3: Marca qualquer estado não marcado com transição vinda de um estado marcado.
 - 4: **until** Nenhum estado seja marcado
 - 5: Se nenhum estado final tiver sido marcado, **aceita**. Caso contrário, **rejeita**.

□

Proposição 17.5 *Seja*

$$E_{Q_{DFA}} = \{\langle A, B \rangle : A \text{ e } B \text{ DFA's e } \mathcal{L}(A) = \mathcal{L}(B)\}.$$

A linguagem $E_{Q_{DFA}}$ é decidível.

Dem. Como a classe das linguagens regulares é fechada para o complemento e intersecção, também é fechada para a diferença simétrica. Ora, duas linguagens são iguais se e só se a sua diferença simétrica é vazia. Assim, podemos substituir a condição $\mathcal{L}(A) = \mathcal{L}(B)$ por

$$\left(\mathcal{L}(A) \cap \overline{\mathcal{L}(B)} \right) \cup \left(\mathcal{L}(B) \cap \overline{\mathcal{L}(A)} \right) = \emptyset.$$

Podemos, então definir a TM que decide $E_{Q_{DFA}}$ da seguinte forma.

M **Input:** $\langle A, B \rangle$ (A e B DFA)

-
- 1: Calcula o DFA C que representa $\left(\mathcal{L}(A) \cap \overline{\mathcal{L}(B)} \right) \cup \left(\mathcal{L}(B) \cap \overline{\mathcal{L}(A)} \right)$.
 - 2: Devolve a resposta do decisor de E_{DFA} aplicado a $\langle C \rangle$.

□

Problemas decidíveis para CFL

Proposição 17.6 *Seja*

$$A_{CFG} = \{\langle G, w \rangle : G \text{ é CFG e } w \in \mathcal{L}(G)\}.$$

A linguagem A_{CFG} é decidível.

Dem. Um decisor para A_{CFG} pode ser definido como a seguinte TM S.

S Input: $\langle G, w \rangle$ (G CFG, w palavra)

1. Converte G numa gramática equivalente G' mas na forma normal de Chomsky.
2. Listam-se todas as derivações de G' com $2^n - 1$ passos, sendo n o tamanho de w .
3. Se alguma destas derivações gera w , **aceita**. Caso contrário, **rejeita**.

□

Proposição 17.7 *Seja*

$$E_{CFG} = \{\langle G \rangle : G \text{ é CFG e } \mathcal{L}(G) = \emptyset\}.$$

A linguagem E_{CFG} é decidível.

Dem. Podemos construir uma TM R que decide E_{CFG} da forma seguinte.

R Input: $\langle G \rangle$ (G CFG)

1. Marca todos os símbolos terminais.
2. Repete até ao processo não marcar mais símbolos.
 - 2.1. Marca qualquer símbolo não terminal A com regra

$$A \rightarrow U_1 U_2 \cdots U_k$$

se todos os símbolos $U_1, U_2, \dots, U_k$ já estiverem marcados.

3. Se o símbolo inicial estiver marcado, **rejeita**. Caso contrário, **aceita**.

□

Um problema reconhecível para TM

Proposição 17.8 *Seja*

$$A_{TM} = \{\langle M, w \rangle : M \text{ é TM e } w \in \mathcal{L}(M)\}.$$

A linguagem A_{TM} é reconhecível.

Dem. Basta ver que a seguinte TM U reconhece A_{TM} .

U Input: $\langle M, w \rangle$ (M TM, w palavra)

1. Executa M com input w .
2. Se M aceitar, **aceita**. Caso contrário, **rejeita**.

□

A TM U da demonstração do teorema anterior, não é particularmente complicada de entender. No entanto é uma TM muito interessante. A máquina U recebe uma descrição de uma outra máquina de Turing, M , assim como de um seu input, w , e simula o funcionamento da máquina M com o input w . À máquina U chamamos normalmente *Máquina de Turing Universal* e esta desempenhará um papel de importância em alguns resultados futuros.

Proposição 17.9 *Seja*

$$E_{TM} = \{\langle M \rangle : M \text{ TM}, \mathcal{L}(M) = \emptyset\}.$$

A linguagem $\overline{E_{TM}}$ é reconhecível.

Dem. Consideremos a seguinte TM que reconhece $\overline{E_{TM}}$.

R Input: $\langle M \rangle$ (M uma TM)

1. Seja $s_1, s_2, \dots$ a sequência de todas as palavras para o alfabeto de M .
2. Para $i \in \mathbb{N}$
 - 2.1. Simula i passos da execução de M com input s_k para $k \leq i$
 - 2.2. Se alguma destas execuções aceitar, para e aceita.

□

17.2 Indecidibilidade

O argumento “diagonal” de Cantor

Quando tratamos de conjuntos finitos, a comparação de cardinalidades não é um problema complicado, mas o caso muda de figura quando tratamos de conjuntos infinitos. Para termos um significado claro de cardinalidade definimo-la como a seguir. Dois conjuntos A e B dizem-se equipotentes (ou equicardinais) se existir uma bijecção $h : A \rightarrow B$.

Claro que se tivermos uma função injectiva $f : A \rightarrow B$ podemos garantir que $|A| \leq |B|$.

A um conjunto que seja equipotente com o conjunto dos números naturais $\mathbb{N}$ dizemos que é *numerável*.

Estamos, pois, em condições de enunciar os seguintes resultados sobre cardinalidade de alguns conjuntos infinitos.

Teorema 17.10 *O conjunto $\mathbb{Z}$ é numerável.*

Dem. Para tal provar basta considerar a seguinte função

$$\begin{aligned} f : \mathbb{Z} &\rightarrow \mathbb{N} \\ 0 &\mapsto 0 \\ i &\mapsto \begin{cases} 2i - 1 & \text{se } i > 0, \\ -2i & \text{se } i < 0. \end{cases} \end{aligned}$$

A função f é trivialmente uma bijecção, pelo que $|\mathbb{N}| = |\mathbb{Z}|$.

□

Teorema 17.11 *O conjunto $\mathbb{Q}$ é numerável.*

Dem. Como $\mathbb{N} \subseteq \mathbb{Q}$ temos que $|\mathbb{N}| \leq |\mathbb{Q}|$. Mostremos que há uma função injectiva $f: \mathbb{Q}^+ \rightarrow \mathbb{N}$. Podemos inscrever todos os elementos de $\mathbb{Q}^+$ numa tabela dispostos como se ilustra a seguir. Podemos percorrer todos os racionais positivos como ilustra a linha a tracejado. Isso constitui uma sucessão de valores racionais $(q_i)_{i \in \mathbb{N}}$ que cobre todos os naturais positivos. Tomemos então $f(q) = \min\{i: q_i = q\}$. A função f é trivialmente injectiva, pelo que $|\mathbb{Q}^+| = |\mathbb{N}|$.

[illegible]

Agora basta mostrar que $|\mathbb{Q}| \leq |\mathbb{N}|$ usando a bijecção

$$\begin{aligned} g: \mathbb{Q} &\rightarrow \mathbb{N} \\ 0 &\mapsto 0 \\ q &\mapsto \begin{cases} 2(f(q) + 1) & \text{se } q > 0; \\ 2(f(q) + 1) + 1 & \text{se } q < 0. \end{cases} \end{aligned}$$

☐

Teorema 17.12 *O conjunto $\mathbb{R}$ não é numerável.*

Dem. Mostremos que o intervalo $[0, 1[\subseteq \mathbb{R}$ não é numerável, o que é suficiente para garantir que $\mathbb{R}$ também não o é. Para isso suponhamos, por absurdo, que $[0, 1[$ era numerável. Então existiria uma função bijetiva $f : [0, 1[\rightarrow \mathbb{N}$. Como a função é bijetiva faz sentido falar em f^{-1} e podemos imaginar a tabela das imagens recíprocas dos sucessivos elementos de $\mathbb{N}$. Para simplificar vamos supor os valores de f^{-1} expressos em binário.

	f ⁻¹ (n)
0	. 0 1 1 0 0 1 1 ...
1	. 1 0 1 1 0 1 0 ...
2	. 1 1 0 1 1 0 1 ...
3	. 0 1 1 0 1 1 0 ...
⋮	⋮ ⋮ ⋮ ⋮ ⋮ ⋮ ⋮

Podemos mostrar que esta função não pode ser bijectiva encontrando um elemento x de $[0, 1[$ que não seja imagem recíproca por f de nenhum natural. Para tal definamos x como o número cujos bits são da seguinte forma: o i -ésimo bit é o complementar do i -ésimo bit da imagem recíproca por f de i . Se x ocorresse na tabela, digamos na posição k , qual seria o valor do seu i -ésimo bit? Deveria ser o complementar do que se encontra na tabela, ou seja, complementar de si próprio, o que é, evidentemente, absurdo. Portanto f não pode ser uma bijecção e $|\mathbb{R}| \geq |[0, 1[| > |\mathbb{N}|$. Logo $\mathbb{R}$ não é numerável. $\square$

Estamos agora em condições de mostrar que nem as máquinas de Turing representam todas as linguagens existentes.

Teorema 17.13 *Há linguagens não reconhecíveis.*

Dem. Vamos primeiro mostrar que o conjunto de todas as TM é numerável. Para isso notemos que, seja qual for o alfabeto Σ , Σ^* é numerável. O facto é evidente, basta observar a seguinte função injectiva $f: \Sigma^* \rightarrow \mathbb{N}$. Suponhamos que $|\Sigma| = k$ então façamos corresponder a cada carácter σ de Σ um diferente valor $b(\sigma)$ em $\{1, 2, \dots, k+1\}$. Façamos $f(\varepsilon) = 0$ e $f(w)$, com $w \in \Sigma^+$, igual ao valor de w transformando cada símbolo pela função b e interpretando o resultado como um número expresso em base $k+2$. A função f é trivialmente injectiva, pelo que $|\Sigma^*| \leq |\mathbb{N}|$. Para a desigualdade contrária basta tomar $\sigma \in \Sigma$ e a função $g: \mathbb{N} \rightarrow \Sigma^*$, com $g(n) = \sigma^n$. Portanto Σ^* é numerável.

Uma TM pode ser representada por uma palavra que descreva completamente os seus elementos constituintes. Como o conjunto de estados de uma TM é finito, e os alfabetos nela intervenientes também o são (por definição de alfabeto), a descrição da sua função de transição assim como dos demais elementos constituem uma palavra (de tamanho finito) para algum alfabeto Σ' . Portanto as palavras de tal alfabeto representam todas as possíveis representações de TM, e portanto o conjunto de todas as TM é numerável.

Seja Σ um alfabeto, vimos que Σ^* é numerável. Uma linguagem com o alfabeto Σ é um qualquer conjunto de palavras de Σ^* . Suponhamos, por absurdo, que o conjunto das linguagens de alfabeto Σ é numerável. Então haveria uma bijecção f entre $\mathbb{N}$ e o conjunto das linguagens com tal alfabeto. Enumeremos todas as palavras de tal alfabeto (o conjunto é numerável) como $w_0, w_1, \dots$. Podemos então imaginar uma tabela que lista todas as linguagens, com a ordem dada pela bijecção f , com a informação de que palavras pertencem a cada uma. Construamos, usando o argumento diagonal de Cantor, uma linguagem que não pode ocorrer na tabela, e que portanto mostra que f não é bijecção. Seja $L = \{w_i : i \in \mathbb{N} \wedge w_i \notin f(i)\}$. Se L estivesse na tabela, ou seja se L fosse imagem de algum natural k , o que se poderia dizer acerca da palavra w_k ?

	w_1	w_2	w_3	$\dots$	w_k	$\dots$
$f(0)$	sim	sim	não	$\dots$	não	$\dots$
$f(1)$	sim	não	sim	$\dots$	sim	$\dots$
$f(2)$	sim	sim	não	$\dots$	sim	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	
$f(k)$	não	sim	sim	$\dots$	?	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	

O absurdo resulta da suposição da existência da bijecção f , pelo que o conjunto das linguagens de alfabeto Σ não é numerável. Portanto existe pelo menos uma linguagem que não é reconhecida por qualquer máquina de Turing. $\square$

Uma primeira linguagem indecidível

Teorema 17.14 *A linguagem A_{TM} é indecidível.*

Dem. Suponhamos, por absurdo, que A_{TM} é decidível. Seja H a TM que decide A_{TM} .

H	Input: $\langle M, w \rangle$ (M TM, w palavra)
1. Se M aceita w , aceita	
2. Caso contrário, rejeita .	

Construamos uma outra TM D que usa H , mas de forma um pouco diferente. Uma descrição $\langle M \rangle$ de uma TM M pode ser vista como uma palavra como outra qualquer. Portanto faz sentido dar como input a uma máquina de Turing a descrição de uma outra máquina.

D	Input: $\langle M \rangle$, (M TM)
1. Se H com input $\langle M, \langle M \rangle \rangle$ aceita, então rejeita	
2. Se H com input $\langle M, \langle M \rangle \rangle$ rejeita, então aceita .	

Resumindo,

$$D(\langle M \rangle) \begin{cases} \text{aceita} & \text{se } M \text{ não aceita } \langle M \rangle, \\ \text{rejeita} & \text{se } M \text{ aceita } \langle M \rangle. \end{cases}$$

Mas o que acontece se executarmos D com input $\langle D \rangle$?

$$D(\langle D \rangle) \begin{cases} \text{aceita} & \text{se } D(\langle D \rangle) \text{ não aceita,} \\ \text{rejeita} & \text{se } D(\langle D \rangle) \text{ aceita.} \end{cases}$$

A TM D é, portanto, um objecto paradoxal, que não pode existir. O absurdo resultou de se ter suposto a existência de H , ou seja que A_{TM} era decidível.

A demonstração da inexistência de D pode mais uma vez ser visto como uma instância do argumento diagonal. Podemos imaginar uma tabela com o comportamento de todas as TM quando lhes é dado como input a descrição de uma outra TM. A TM D é definida como, quando com input $\langle M \rangle$, tendo o comportamento complementar à TM $M(\langle M \rangle)$. O absurdo resulta quando queremos averiguar qual o comportamento de D quando tem o input $\langle D \rangle$, pois nesse caso o comportamento de $D(\langle D \rangle)$ está definido como o complementar de $D(\langle D \rangle)$. $\square$

Uma linguagem não reconhecível

Definição 17.15 *Diz-se que uma linguagem L é co-reconhecível se for o complementar de uma linguagem $\bar{L}$ reconhecível.*

Teorema 17.16 *Uma linguagem é decidível se e só se é reconhecível e co-reconhecível.*

Dem. Se A é decidível então é evidente que tanto A como $\bar{A}$ são reconhecíveis, podendo para tal usar a TM que decide A . No sentido contrário. Se tanto A como $\bar{A}$ são reconhecíveis, seja M_1 o reconhecedor de A e M_2 o reconhecedor de $\bar{A}$. Podemos construir uma TM M que decide A da seguinte forma:

M **Input:** $\langle w \rangle$, (w palavra)

-
1. Executa as máquinas M_1 e M_2 , em paralelo, com input w .
 2. Se M_1 aceita, então **aceita**. Se M_2 aceita, então **rejeita**.

Executar as duas máquinas em paralelo significa executar uma máquina com duas fitas, uma para cada máquina, uma para a máquina M_1 e outra para a máquina M_2 . É executado um passo de cada máquina, alternadamente, até que alguma delas pare e aceite a palavra w . $\square$

Corolário 17.17 A linguagem $\overline{A_{TM}}$ é não reconhecível.

Dem. Sabemos que A_{TM} é reconhecível (Teorema 17.8). Se $\overline{A_{TM}}$ fosse reconhecível também, então seria decidível. Logo $\overline{A_{TM}}$ é não reconhecível. $\square$

Mais linguagens não decidíveis

O problema da paragem

Teorema 17.18 Seja

$$HALT_{TM} = \{ \langle M, w \rangle : M \text{ é uma TM e } M \text{ para com input } w \}.$$

Então $HALT_{TM}$ não é decidível.

Dem. A demonstração é feita por contradição. Vamos mostrar que se existisse um decisor para $HALT_{TM}$ então seria possível construir um decisor para A_{TM} , o que como sabemos, é impossível (Teorema 17.14). Suponhamos então que a TM R decidia $HALT_{TM}$. Então poderíamos construir a TM S da seguinte maneira.

S **Input:**

-
1. Executa R com input $\langle M, w \rangle$.
 2. Se R rejeita, então **rejeita**.
 3. Se R aceita, então simula M até parar.
 4. Se M aceita, **aceita**. Se M rejeita, **rejeita**.

Claramente S decide A_{TM} , o que é absurdo. Portanto $HALT_{TM}$ não é decidível. $\square$

Proposição 17.19 Seja

$$E_{TM} = \{ \langle M \rangle : M \text{ é TM e } \mathcal{L}(M) = \emptyset \}.$$

Então E_{TM} é não decidível.

Dem. Mostremos que se E_{TM} fosse decidível então também A_{TM} o seria. Suponhamos, por absurdo, que R uma TM decidia E_{TM} . Podemos então construir S que decida A_{TM} .

S Input: $\langle M, w \rangle$, (M TM, w palavra) 1. Com M e w constroi: M_1 Input: x 1 Se $x \neq w$ rejeita. 2 Se $x = w$, executa M com input w e aceita se M aceita w. 2. Executa R com input $\langle M_1 \rangle$. 3. Se R aceita, rejeita. Se R rejeita, aceita.

S seria um decisor para A_{TM} , pelo que E_{TM} é indecidível. □

Proposição 17.20 *Seja*

$$REGULAR_{TM} = \{ \langle M \rangle : M \text{ é TM e } \mathcal{L}(M) \text{ é regular} \}.$$

Então $REGULAR_{TM}$ é indecidível.

Dem. Mais uma vez provamos que se R fosse um decisor para $REGULAR_{TM}$ então poderíamos construir um decisor S para A_{TM}

Seja S a TM descrita da seguinte forma.

S Input: $\langle M, w \rangle$, (M uma TM, w palavra) 1. Com M e w constrói: M_2 Input: x 1 Se x tem a forma $0^n 1^n$, aceita. 2 Senão, executa M com input w e aceita se M aceita w. 2. Executa R com input $\langle M_2 \rangle$. 3. Se R aceita, aceita. Se R rejeita, rejeita.

M_2 aceita palavras $0^n 1^n$, o que não constitui uma linguagem regular, mas se M aceitar w , então M_2 passa a aceitar Σ^* , que é regular. Isto é suficiente para criar S . Note-se que M_2 nunca é executada. É construída somente para a sua representação ser passada a R como input. Como sabemos que A_{TM} é indecidível $REGULAR_{TM}$ é indecidível. □

Exercício

82 — Mostrar que a classe das linguagens decidíveis não é fechada para homomorfismos.

O teorema de Rice

Da mesma forma se poderia mostrar que determinar se uma TM representa uma CFL, ou uma linguagem finita são problemas indecidíveis. Para mostrar tais factos há um resultado geral que torna estes outros resultados consequências imediatas.

Definição 17.21 *Uma propriedade P (um conjunto de linguagens) diz-se não trivial se for uma propriedade intrínseca das linguagens (e não da TM em particular que a representa), contiver alguma linguagem mas não contiver todas.*

O teorema de Rice (17.22) estabelece que todas as propriedades não triviais são indecidíveis.

Teorema 17.22 (Teorema de Rice) *Seja P uma propriedade não trivial de linguagens. O problema de determinar se a linguagem representada por uma TM tem essa propriedade é indecidível.*

Dem. Suponhamos R_P decisor para a propriedade P e L_P a sua linguagem, ou seja

$$L_P = \{\langle M \rangle : M \text{ é uma TM e } \mathcal{L}(M) \in P\}.$$

Então construímos uma TM S que vai ser decisor de A_{TM} . Seja $T_\emptyset$ uma TM que rejeita todo o input, i.e. $\mathcal{L}(T_\emptyset) = \emptyset$. Podemos supor que $\langle T_\emptyset \rangle \notin L_P$, se não fosse esse o caso consideraríamos $\bar{P}$. Como P é não trivial, existe uma TM T com $\langle T \rangle \in L_P$. Seja então S uma TM definida do seguinte modo.

S Input: $\langle M, w \rangle$, (M TM, w palavra) 1. Com M e w constrói: M_w Input: x 1 Simula M com input w. Se M pára e rejeita, então rejeita. 2 Se M aceita então simula T com input x e se T aceita, aceita. 2. Usa R_P para determinar se $\langle M_w \rangle \in L_P$, e procede de acordo com a resposta. 3. Se R aceita, aceita. Se R rejeita, rejeita.
--

A TM M_w simula T se M aceita w . Portanto $\mathcal{L}(M_w) = \mathcal{L}(T)$ se M aceita w e $\emptyset$ no caso contrário. Portanto $\langle M_w \rangle \in L_P$ se e só se M aceita w . Portanto S decide A_{TM} e então R_P não pode existir, pelo que P é indecidível. $\square$

Proposição 17.23 *Seja*

$$EQ_{TM} = \{\langle M_1, M_2 \rangle : M_1, M_2 \text{ TM e } \mathcal{L}(M_1) = \mathcal{L}(M_2)\},$$

então EQ_{TM} é indecidível.

Dem. Podemos mostrar que decidir EQ_{TM} implicaria decidir E_{TM} . Seja R o decisor de EQ_{TM} , construímos uma TM S .

S **Input:** $\langle M \rangle, \langle M \text{ TM} \rangle$

1. Executa R com input $\langle M, M_\emptyset \rangle$. Em que $\mathcal{L}(M_\emptyset) = \emptyset$.
2. Procede de acordo com a resposta.

Se R decide $E_{Q_{TM}}$ então S decide E_{TM} , pelo que $E_{Q_{TM}}$ é indecidível.

□